

QuadrupolarDFT

User Manual

A practical route from crystal structure to NQR parameters

This manual leads you from a crystal structure to electric-field-gradient (EFG) tensors, quadrupolar coupling constants, and zero-field NQR transition frequencies. It begins with a first static calculation, then builds toward finite-temperature, displacement, and strain workflows. ABINIT provides the efficient production path; Elk provides an all-electron accuracy check when the near-nucleus EFG needs closer scrutiny.

Contents

I	Getting Started	1
1	Start Here: From a Structure to One NQR Prediction	2
1.1	This manual and the Beginner's Guide	2
1.2	The four objects you will work with	2
1.3	Install and run a tensor-only first example	3
1.4	Choose the smallest workflow that answers your question	4
1.5	A result is ready only after four checks	4
2	Choosing a Calculation Strategy: Accuracy, Time, and Evidence	5
2.1	ABINIT and Elk are different tools, not rival answers	5
2.2	Budget the workflow in solves, not a universal wall time	5
2.3	A practical two-tier protocol	6
2.4	What the NaNO_2 comparison teaches	6
2.5	Diagnose disagreement before doing more computation	6
3	Scope and Physical Background	8
3.1	Why a finite-temperature correction is needed	8
4	Installation and Conventions	10
4.1	Installation	10
4.2	Tensor and unit conventions	10
4.3	Link to the spin-dynamics simulator	10
II	Core Calculations	11
5	Static EFG Workflow	12
6	Finite-Temperature EFG	13
6.1	Harmonic vibrational averaging	13
6.2	The analytic Bayer limit	13

III	Advanced Workflows	15
7	The DFPT Finite-Displacement Workflow	16
7.1	Normal-coordinate convention	16
7.2	Stage 0 — structure relaxation (optional)	16
7.3	Stage 1 — phonons	17
7.4	Stage 2 — displaced EFG runs	17
7.5	Stage 3 — collect	18
7.6	Running ABINIT in parallel	18
7.7	Convergence of η and the asymmetry gap	18
8	Finite-Strain EFG Drive Couplings	21
8.1	Glycine polymorph guard	21
8.2	Static and strained ABINIT jobs	21
8.3	First-pass glycine result	22
9	Worked Example: NaNO_2 ^{14}N	23
IV	Evidence and Reference	25
10	Validation Against the Measured Database	26
11	Public API Reference	27
11.1	quadrupolar_dft.tensors / quadrupolar / abinit	27
11.2	quadrupolar_dft.thermal	27
11.3	quadrupolar_dft.vibrational	27
11.4	quadrupolar_dft.finite_displacement / abinit_phonon	27
11.5	quadrupolar_dft.strain_coupling	27
11.6	quadrupolar_dft.relax	28
12	Limits of the Current Workflow	29

PART I

Getting Started

1 Start Here: From a Structure to One NQR Prediction

In this chapter. You will learn what the workspace does, which result to calculate first, and how to check that a result is ready for scientific use. No prior DFT workflow experience is assumed.

The shortest useful QuadrupolarDFT workflow has three steps:

1. start from a crystal structure whose origin and temperature are known;
2. calculate an EFG tensor with ABINIT; and
3. convert that tensor into C_Q , η , and NQR transition frequencies.

That first static prediction is the foundation for every later calculation in this manual. Do it before attempting phonons, temperature corrections, or strain derivatives.

1.1 This manual and the Beginner's Guide

This is a *task-oriented* manual. It tells you which files to prepare, which commands to run, what the output means, and where to stop and check your work. The companion `docs/DFT_for_Spin_Dynamics_Beginners_Guide.pdf` explains the ideas behind those tasks: self-consistent fields, k -point meshes, basis sets, pseudopotentials, all-electron methods, phonons, and parallel execution.

You do not need to read the whole Beginner's Guide first. Follow this manual and use the Guide when a concept is unfamiliar. In particular:

- read its introductory DFT and SCF sections before editing an ABINIT input for the first time;
- read its convergence discussion before trusting a reported EFG;
- read its PAW-versus-all-electron discussion before comparing ABINIT and Elk; and
- read its phonon and ABINIT DDB/ANADDB sections before Chapter 7.

1.2 The four objects you will work with

Keeping four objects separate makes the workflow much easier to reason about:

Structure

Lattice vectors, atomic coordinates, species, and the experimental or relaxed geometry from which they came.

Electronic-structure calculation

The ABINIT or Elk input, pseudopotentials or all-electron basis controls, exchange–correlation functional, k -point mesh, and convergence thresholds.

EFG tensor

A symmetric 3×3 tensor at a particular nucleus. This is the direct DFT result and should be retained, not only its principal value.

Magnetic-resonance observables

C_Q , η , principal axes, and transition frequencies derived using an isotope's spin and quadrupole moment.

If a prediction is surprising, this separation tells you where to look: structure, electronic structure, tensor interpretation, or nuclear data.

1.3 Install and run a tensor-only first example

From the QuadrupolarDFT directory, create and activate a virtual environment, then install the workspace:

```
python -m venv .venv
# Linux/macOS:
. .venv/bin/activate
# Windows PowerShell:
# .venv\Scripts\Activate.ps1
python -m pip install -e .
```

The package can analyze a tensor without ABINIT or Elk. Save the following as `quick_start.py`; it is a useful installation check:

```
from quadrupolar_dft import (
    EFGTensor, coupling_constant_hz, nqr_frequencies_hz
)

efg = EFGTensor.from_components([
    [-0.236543, 0.864057, 0.0],
    [0.864057, -0.236543, 0.0],
    [0.0, 0.0, 0.473086],
])
cq = coupling_constant_hz(
    efg.vzz_si, quadrupole_moment_barns=-0.02558
)
print("eta =", efg.eta)
print("Cq (MHz) =", cq / 1e6)
print("lines (Hz) =", nqr_frequencies_hz(
    spin=2.5, cq_hz=cq, eta=efg.eta
))
```

Run it with `python quick_start.py`. The tensor is interpreted in ABINIT atomic units by default; this convention is explained in Chapter 4. For the complete worked calculation, use the provided `NaNO2` input in `examples/abinit/`; Chapter 9 walks through the files and the expected checks.

What this command does not prove. A successful Python example checks parsing and tensor-to-NQR conversion. It does not show that a DFT basis, k -point mesh, structure, or pseudopotential is accurate. Those are physical convergence questions, not software installation tests.

1.4 Choose the smallest workflow that answers your question

Question	Start with	Add only when needed
What lines should this structure produce?	Static ABINIT EFG	Elk cross-check or finite-temperature correction
Why does calculation disagree with experiment?	Structure audit and ABINIT convergence	Elk on the same geometry and functional
How do lines change with temperature?	Converged static reference	ABINIT DFPT and finite displacements
How does strain drive or shift a transition?	Converged static reference	Thirteen-job finite-strain set

1.5 A result is ready only after four checks

Before quoting a frequency, confirm:

1. **Identity:** the reported tensor belongs to the intended atom and isotope.
2. **Numerics:** total energy and EFG components are stable to tighter cutoffs, denser k -point sampling, and stricter SCF convergence.
3. **Conventions:** principal values are ordered by magnitude and the sign and units of Q , V_{zz} , and C_Q are recorded.
4. **Provenance:** structure, code version, functional, pseudopotential or all-electron settings, and input/output files are saved.

The remaining chapters turn those checks into repeatable workflows.

2 Choosing a Calculation Strategy: Accuracy, Time, and Evidence

In this chapter. You will decide when fast PAW calculations are sufficient, when an all-electron check is worth its cost, and how to interpret disagreement between ABINIT, Elk, and experiment.

EFGs are unusually sensitive to the electron density close to a nucleus. That makes the choice between a PAW calculation and an all-electron calculation more important here than it may be for total energies or relaxed lattice constants. In this repository, ABINIT and Elk therefore have complementary roles.

2.1 ABINIT and Elk are different tools, not rival answers

	ABINIT PAW	Elk LAPW all-electron
Best role	Production calculations, convergence surveys, relaxation, DFPT, temperature ensembles, and strain derivatives	High-fidelity static reference for selected structures and diagnosis of PAW/core sensitivity
Why it is faster	Smooth pseudo-wavefunctions and efficient k -point parallelism	—
Main accuracy risk	The PAW dataset may reconstruct the near-nucleus density poorly for the EFG	Basis, muffin-tin radii, local orbitals, and k points must still be converged
Main time cost	Many moderate-cost solves in advanced workflows	Dense generalized eigenproblems whose cost rises steeply with the LAPW basis
Repository support	Complete static, relaxation, phonon, displacement, and strain path	Static reference and convergence examples

The label *all-electron* does not mean *automatically exact*. Elk still depends on the crystal structure, functional, k -point mesh, LAPW cutoff, muffin-tin radii, local orbitals, and SCF tolerance. Its value is that it removes the PAW dataset as one major source of uncertainty.

2.2 Budget the workflow in solves, not a universal wall time

A statement such as *Elk is ten times slower* would be misleading: timing depends strongly on structure, hardware, linear-algebra libraries, parallel layout, and convergence settings. A more portable estimate counts independent electronic-structure solves:

- a static prediction needs roughly one converged solve, plus cutoff and k -point checks;
- a convergence survey needs a baseline and every off-baseline point;
- a finite-displacement temperature calculation needs the phonon work, one equilibrium EFG, and two EFG calculations for each retained mode;
- the six-component central finite-strain workflow needs thirteen EFG calculations: one equilibrium and positive/negative strain for each component; and

- an Elk check should normally cover a few representative structures, with its own `rgkmax` and k -point checks.

This arithmetic explains the repository strategy: use ABINIT for broad or derivative-heavy work, and spend the more expensive all-electron calculation where it changes the strength of the scientific conclusion.

2.3 A practical two-tier protocol

1. **Audit the structure.** Record its source, temperature, occupancy, symmetry choices, and whether it is experimental or relaxed.
2. **Converge the ABINIT EFG.** Vary cutoff and k -point density, monitoring the full tensor and η , not only total energy.
3. **Compare with the available evidence.** Consider experimental uncertainty, temperature, polymorph, isotope, and line assignment.
4. **Run Elk on the same geometry and functional** when the result is publication-critical, unexpectedly discrepant, or sensitive to a small difference between transverse EFG components.
5. **Escalate selectively.** Use the diagnosis below to decide whether to improve the PAW dataset, structure, functional, or thermal model.

2.4 What the NaNO_2 comparison teaches

The repository's NaNO_2 example holds the PBE functional and ICSD 82857 geometry fixed. The measured and calculated nitrogen results are:

Source	η	$ C_Q $ (MHz)
Experiment	approximately 0.38	5.49
ABINIT PAW	0.112	5.03
Elk LAPW, all-electron	approximately 0.333	approximately 5.94

Elk's much better asymmetry on the same structure and functional points to the selected nitrogen PAW dataset's reconstruction of the transverse, near-nucleus EFG as the main limitation. It does *not* prove that Elk will always be closer to experiment, nor that thermal and structural effects are unimportant in other systems.

The Elk convergence example is also a time warning. Results were already stable between `rgkmax`=7 and 8 and between the tested k meshes, while a larger automatic basis became both much more expensive and numerically problematic after muffin-tin radii changed. Use optimized BLAS and no more MPI ranks than useful irreducible k points; consult the Beginner's Guide for why that parallel layout matters.

2.5 Diagnose disagreement before doing more computation

ABINIT and Elk disagree on the same geometry and functional.

Suspect PAW reconstruction or incomplete convergence first. Converge both representations and, if possible, test an alternative PAW dataset.

ABINIT and Elk agree, but both disagree with experiment.

Look next at structure, polymorph, temperature, disorder, exchange–correlation functional, nuclear quadrupole moment, and spectral assignment.

 $|C_Q|$ is stable but η is not.

Inspect the tensor components directly. When V_{xx} and V_{yy} are close, a small absolute error can cause a large relative change in η .

Only the sign disagrees.

NQR transition frequencies often do not determine the sign of C_Q . Check tensor and nuclear-moment conventions before treating it as a physical discrepancy.

Decision rule. Use ABINIT by default. Add Elk when a near-nucleus accuracy question could change the interpretation, and compare on the same structure and functional. The useful result is not which code wins, but which source of uncertainty the comparison isolates.

3 Scope and Physical Background

In this chapter. You will connect the calculated EFG tensor to quadrupolar Hamiltonians and see why a static 0 K result may need a temperature correction.

QuadrupolarDFT addresses one link in the magnetic-resonance modelling chain: it turns an *ab initio* electronic-structure result into the parameters that control a nuclear quadrupole resonance (NQR) experiment.

Readers new to electronic-structure theory may want to start with the companion docs/DFT_for_Spin_Dynamics_Beginners_Guide.pdf, which builds up the background this manual assumes — the SCF loop, k -points and basis sets, PAW versus all-electron treatments, and phonons — from a spin-dynamics standpoint.

A nucleus with spin $I \geq 1$ carries an electric quadrupole moment Q that couples to the electric-field-gradient (EFG) tensor $V_{ij} = \partial^2 V / \partial x_i \partial x_j$ produced by the surrounding charge distribution. The EFG is symmetric and, away from the nucleus, traceless (Laplace's equation), so in its principal-axis system it is described by two numbers: the largest-magnitude component V_{zz} and the asymmetry parameter

$$\eta = \frac{V_{xx} - V_{yy}}{V_{zz}}, \quad |V_{zz}| \geq |V_{yy}| \geq |V_{xx}|, \quad 0 \leq \eta \leq 1. \quad (3.1)$$

The quadrupolar coupling constant follows from V_{zz} and the nuclear quadrupole moment,

$$C_Q = \frac{e Q V_{zz}}{h}, \quad (3.2)$$

and the zero-field NQR transition frequencies follow from C_Q , η and the spin I by diagonalizing the quadrupole Hamiltonian

$$\frac{H_Q}{h} = \frac{C_Q}{4I(2I-1)} [3I_z^2 - I(I+1) + \eta(I_x^2 - I_y^2)]. \quad (3.3)$$

For spin $I = 1$ (e.g. ^{14}N) this gives the familiar triplet

$$\nu_{\pm} = \frac{3}{4}C_Q \left(1 \pm \frac{\eta}{3}\right), \quad \nu_0 = \nu_+ - \nu_- = \frac{1}{2}C_Q \eta. \quad (3.4)$$

3.1 Why a finite-temperature correction is needed

A static DFT calculation evaluates V_{ij} at a single, fixed nuclear configuration at 0 K. Measured NQR frequencies, however, are taken at finite temperature and are strongly temperature dependent: in molecular crystals the lines typically *fall* as temperature rises (the Bayer law). Two physically distinct effects separate the static DFT number from the measurement:

1. **Thermal expansion / geometry.** The lattice constants and internal coordinates at temperature T differ from the relaxed 0 K cell. This is a static effect: it is captured by evaluating the EFG on a temperature-appropriate structure.

2. **Vibrational averaging.** The nucleus samples a thermal distribution of positions through the phonons. The measured EFG is the tensor averaged over that motion, including zero-point motion, which is present even at 0 K. For molecular crystals this dynamic term usually dominates: low-frequency librations reorient the EFG principal axes and reduce the line frequencies.

QuadrupolarDFT provides the static layer (Chapter 5) and a harmonic treatment of the vibrational term (Chapters 6–7).

4 Installation and Conventions

In this chapter. You will install the Python helpers and fix the tensor, unit, and nuclear-data conventions that every later result depends on.

4.1 Installation

The package depends only on NumPy. From the `QuadrupolarDFT/` directory:

```
python -m pip install -e .
python -m pip install -e ".[dev]"      # adds ruff for linting
python -m unittest discover -s tests
```

On a system whose Python is externally managed (PEP 668), the package can also be used without installation by setting `PYTHONPATH` to the `src/` directory, since NumPy is the only requirement:

```
export PYTHONPATH="$PWD/src"
python3 examples/abinit/efg_temperature.py --help
```

4.2 Tensor and unit conventions

`EFGTensor` stores the symmetric tensor in SI units (V/m^2) and sorts the principal components as $|V_{zz}| \geq |V_{yy}| \geq |V_{xx}|$. It accepts input in ABINIT atomic units (the default), in SI, or in the $10^{21} \text{V}/\text{m}^2$ scale, and exposes `vzz_si` and `eta`. Tensor averaging (`average_tensors`) operates on the tensor components and only then diagonalizes the result, so that asymmetry information survives the average.

4.3 Link to the spin-dynamics simulator

The companion `spin_dynamics` package parameterizes a quadrupolar site by its $\eta = 0$ transition frequency ν_Q . This is related to C_Q by the prefactor of Eq. (3.3):

$$\nu_Q = C_Q \cdot \frac{d}{4I(2I-1)} = \begin{cases} \frac{3}{4} C_Q & I = 1, \\ \frac{1}{2} C_Q & I = \frac{3}{2}, \end{cases} \quad (4.1)$$

with $d = 3$ for spin 1 and $d = 6$ for spin 3/2. The cross-project `mr_integration` layer uses this mapping to feed DFT parameters into the simulator and to compare predictions against the measured NQR database.

PART II

Core Calculations

5 Static EFG Workflow

In this chapter. You will prepare a production ABINIT EFG calculation, parse its tensor, convert it to NQR parameters, and test numerical convergence.

The static path produces C_Q , η and the NQR lines from an ABINIT EFG calculation.

1. `format_abinit_efg_block` emits the `nucefg/quadmom` input lines that request an EFG calculation (PAW datasets are required; norm-conserving pseudopotentials are not suitable).
2. ABINIT is run on the input (see `examples/abinit/run_nano2_efg_wsl.sh`).
3. `parse_abinit_efg` parses the `ElectricFieldGradientCalculation` blocks of the output into `AbinitEFGRecord` objects, one per atom, carrying C_Q , η , the principal values/axes, and the total EFG tensor.
4. `coupling_constant_hz` and `nqr_frequencies_hz` convert a tensor and a nuclear quadrupole moment into C_Q and the unique zero-field transition frequencies (Eqs. (3.2)–(3.3)).

`examples/parse_abinit_efg.py` demonstrates parsing an output file. The EFG components, not the total energy, must be converged against `ecut`, `pawecutdg` and the k -point mesh.

Static-workflow checkpoint. Save the complete tensor, atom index, isotope and quadrupole moment, ABINIT input/output, PAW dataset identifiers, code version, structure source, and a convergence table. A single printed C_Q is not a reproducible DFT result.

6 Finite-Temperature EFG

In this chapter. You will see how phonon motion changes a static EFG and how the package converts mode curvatures into temperature-dependent NQR lines.

This chapter explains the model and its outputs. If you only need to understand a supplied set of mode curvatures, read it straight through. If you need to calculate those curvatures, continue to the staged workflow in Chapter 7. The Beginner’s Guide gives the more detailed phonon background.

6.1 Harmonic vibrational averaging

Expanding the EFG tensor to second order in the mass-weighted phonon normal coordinates Q_k and averaging over the harmonic distribution gives

$$\langle V_{ij} \rangle(T) = V_{ij}^{\text{eq}} + \frac{1}{2} \sum_k \frac{\partial^2 V_{ij}}{\partial Q_k^2} \langle Q_k^2 \rangle(T), \quad (6.1)$$

where the mean-square amplitude of mode k (angular frequency ω_k) is

$$\langle Q_k^2 \rangle(T) = \frac{\hbar}{2\omega_k} \coth\left(\frac{\hbar\omega_k}{2k_B T}\right). \quad (6.2)$$

The $\coth$ factor equals $2n(\omega_k, T) + 1$ with n the Bose–Einstein occupation. It tends to 1 as $T \rightarrow 0$ (pure zero-point motion — which is why the static EFG is never exactly what is measured) and to $2k_B T / \hbar\omega_k$ in the classical high-temperature limit. These quantities live in `quadrupolar_dft.thermal`.

Tensor-frame averaging. The average in Eq. (6.1) is taken on the tensor components in a fixed crystal frame; the result is diagonalized only afterwards. Librational motion reorients the principal-axis system, so averaging V_{zz} or η directly would discard exactly that effect. As a consequence η is itself temperature dependent. `thermally_averaged_efg` performs the average and removes any residual trace (the EFG is traceless by Laplace’s equation; the finite-difference curvatures of Section 7.1 amplify rounding noise into a spurious trace, which is projected out). `efg_temperature_sweep` returns $C_Q(T)$, $\eta(T)$ and the lines at each requested temperature.

6.2 The analytic Bayer limit

Where per-mode EFG curvatures are not available, the temperature *form* can be validated directly against a measured line series with the single-libration Bayer–Kushida model

$$\nu(T) = \nu_0 \left[1 - a \coth\left(\frac{\hbar\omega}{2k_B T}\right) \right], \quad (6.3)$$

where a is a small dimensionless librational amplitude. `fit_bayer_single_mode` fits (ν_0, a, ω) to measured $\nu(T)$ data with a NumPy-only routine: for a fixed ω the model is linear in ν_0 and $\nu_0 a$, so a grid scan over ω with a linear least-squares solve at each point suffices.

Interpretation limit. A good Bayer fit shows that a line series has the expected thermal form. It does not identify a unique phonon mode or replace a converged first-principles vibrational calculation.

PART III

Advanced Workflows

7 The DFPT Finite-Displacement Workflow

In this chapter. You will run the staged ABINIT phonon and displacement workflow, collect its EFG tensors, and validate the numerical derivatives before using them for temperature prediction.

The mode curvatures $\partial^2 V_{ij} / \partial Q_k^2$ in Eq. (6.1) come from displacing the structure along each phonon mode and recomputing the EFG. Because the DFT runs are expensive and run outside the package, the workflow separates into three stages with a calculation between each, preceded by an optional structure-relaxation stage (Stage 0) that puts the geometry at an energy minimum first. The CLI `examples/abinit/efg_temperature.py` drives all of them.

Stage	Python action	Expensive external calculation
0 (optional)	write/collect relaxation	ABINIT relaxation
1	write phonon inputs	ABINIT DFPT, then ANADDB
2	write displaced inputs	one ABINIT EFG per job
3	collect and average	none

Before Stage 1. Finish the static convergence study first. Phonons and displacements multiply the cost of every weak choice in the base input. Copy the converged input into a new run directory and preserve it unchanged as the workflow reference.

7.1 Normal-coordinate convention

A phonon eigenvector ε_k is mass-weighted and normalized ($\sum |\varepsilon_k|^2 = 1$). A normal-coordinate step δQ displaces atom i (mass m_i) by

$$\Delta \mathbf{r}_i = \delta Q \frac{\varepsilon_{k,i}}{\sqrt{m_i}}, \quad (7.1)$$

and the curvature is the central difference of the EFG tensors at $0, \pm \delta Q$,

$$\frac{\partial^2 V_{ij}}{\partial Q_k^2} \approx \frac{V_{ij}(+\delta Q) - 2V_{ij}(0) + V_{ij}(-\delta Q)}{\delta Q^2}. \quad (7.2)$$

These units are consistent with $\langle Q_k^2 \rangle$ from Eq. (6.2), so the product in Eq. (6.1) is an EFG. `normal_coordinate_step_si` chooses δQ from a target maximum Cartesian displacement (default 0.05 Å).

7.2 Stage 0 — structure relaxation (optional)

The harmonic phonon model is meaningful around a mechanically stable reference. If the chosen structure has substantial residual forces, imaginary Γ -point modes and biased equilibrium parameters can follow. An experimental structure may still be the right static comparison, so retain it and document any relaxed

structure as a distinct model rather than silently replacing one with the other. `efg_temperature.py relax` strips the EFG keywords from a converged static input and prepends a BFGS ionic-relaxation block (`ionmov 2, optcell 0, tolmx`); `run_relax_wsl.sh` runs ABINIT. `efg_temperature.py relax-collect` then reads the relaxed geometry from the output footer (`-outvars:echovaluesofvariablesaftercomputation`, the same block `abipy` reads) and writes a relaxed static EFG input.

```
python3 examples/abinit/efg_temperature.py relax \
  --base examples/abinit/nano2_efg.abi --out runs/nano2_relax
bash examples/abinit/run_relax_wsl.sh runs/nano2_relax
python3 examples/abinit/efg_temperature.py relax-collect \
  --base examples/abinit/nano2_efg.abi --abo runs/nano2_relax/relax.abo \
  --out runs/nano2_relax
```

The product, `relaxed.abi`, is an ordinary static EFG input (EFG keywords intact) at the relaxed geometry; pass it as `--base` to Stages 1–2 in place of the hand-written input. The relaxation is internal-coordinates-only: the cell is held at the input value (`optcell 0`). For molecular crystals this is usually preferable to a full cell relaxation, which under GGA tends to over-expand the lattice and can worsen the EFG relative to the measured geometry. Temperature-dependent cell expansion is not included in the present harmonic finite-displacement workflow; treat it as a separate structural contribution.

7.3 Stage 1 — phonons

`efg_temperature.py phonon` writes a two-dataset ABINIT input (ground state plus a Γ -point DFPT response) and the matching `anaddb` input, derived from a converged static EFG input. `run_phonon_wsl.sh` runs ABINIT, selects the DFPT derivative database (the largest `*_DDB`), and runs `anaddb` to produce the phonon frequencies and eigenvectors.

```
python3 examples/abinit/efg_temperature.py phonon \
  --base examples/abinit/nano2_efg.abi --out runs/nano2_ph
bash examples/abinit/run_phonon_wsl.sh runs/nano2_ph
```

The generated DFPT input is a starting template. Two settings are required for PAW + GGA phonons and are emitted automatically: `iscf2 7` (the response driver requires `iscf  $\leq 9$`) and `pawxcdev 0` (the GGA exchange correlation kernel is not implemented in the development scheme). Converge the tolerances and k -mesh for production use.

7.4 Stage 2 — displaced EFG runs

`efg_temperature.py displace` parses the `anaddb` eigenvectors (`parse_anaddb_modes`; acoustic and imaginary modes are dropped, and modes are paired with frequencies by mode number so that gaps in the eigenvector output do not misalign them), generates one displaced EFG input per $\pm$ mode plus a manifest, and writes positions as fractional xred coordinates. `run_finite_displacement_wsl.sh` runs ABINIT on every input.

```
python3 examples/abinit/efg_temperature.py displace \
  --base examples/abinit/nano2_efg.abi --anaddb runs/nano2_ph/anaddb.out \
  --target 2 --max-modes 6 --out runs/nano2_disp
bash examples/abinit/run_finite_displacement_wsl.sh runs/nano2_disp
```

--target is the 0-based index of the resonant nucleus; --max-modes restricts the calculation to the lowest-frequency librations, which dominate the temperature dependence. If the anadddb eigenvector layout differs from the parser's expectation, -modes modes.json supplies frequencies and eigenvectors directly.

7.5 Stage 3 — collect

efg_temperature.py collect parses the displaced EFG outputs, central differences the mode curvatures, runs the temperature sweep, and reports $C_Q(T)$, $\eta(T)$, the lines, and $d\nu/dT$.

```
python3 examples/abinit/efg_temperature.py collect \
    --workdir runs/nano2_disp --temperatures 0,77,150,300 --quadmom 0.02044
```

Do not skip the derivative check. Repeat selected modes with a smaller and larger displacement. The resulting curvature should be stable: too small a step amplifies SCF noise, while too large a step samples anharmonic response. Inspect plus/minus tensor changes, imaginary modes, and the residual trace before interpreting a temperature slope.

7.6 Running ABINIT in parallel

The phonon and EFG runs dominate the wall-clock cost, and ABINIT (MPI-built) parallelizes them efficiently over k -points. All the runner scripts source a small shared helper, examples/abinit/abinit_parallel.sh, and honor the ABINIT_NP environment variable:

- ABINIT_NP unset (or 1) — serial.
- ABINIT_NP= N — launch mpirun -np N abinit.
- ABINIT_NP=auto — choose N automatically (see below) and fall back to serial if no MPI runtime is found.

With the default k -point parallelism (para1_kgb 0), efficiency is good when the number of processes divides the number of k -points in the IBZ. The helper function abinit_optimal_np encodes this: it returns the largest divisor of n_{kpt} that is $\leq$ the core count. For the NaNO_2 cell ($n_{\text{kpt}} = 36$) on a 20-core machine that is $N = 18$ (two k -points per process), which scales near-ideally ($\sim 18\times$). auto detects n_{kpt} from an existing .abo or a quick abinit -dry-run, and the core count from nproc. Override the launcher with ABINIT_MPIRUN (e.g. mpiexec, or mpirun -oversubscribe). The driver nano2_relaxation_study.sh uses auto by default; pass --np N to pin a value or -np 1 to force serial.

Parallelization changes only how ABINIT is launched, not the inputs, so it leaves all computed quantities (and any relaxed-vs-unrelaxed comparison) unchanged.

7.7 Convergence of η and the asymmetry gap

The coupling $C_Q \propto V_{zz}$ tracks the largest EFG eigenvalue and converges quickly, but the asymmetry $\eta = (V_{xx} - V_{yy})/V_{zz}$ is a normalized *difference* of the two smaller eigenvalues. It therefore converges

much more slowly with the PAW double-grid cutoff `pawcutdg` and the k -mesh `ngkpt`, and a calculation can reproduce C_Q while still reporting a too-axial (low) η simply because the transverse tensor components are not yet converged. `examples/abinit/efg_convergence.py` isolates that effect: it sweeps `ecut`, `pawcutdg`, and `ngkpt` one knob at a time around the baseline read from a base input, then tabulates η (and C_Q) per knob with successive deltas and a converged flag.

```
PYTHONPATH=src python3 examples/abinit/efg_convergence.py generate \
--base examples/abinit/nano2_efg.abi --target 2 \
--ecut 25,30,35 --pawcutdg 50,60,80,100 --ngkpt 4,6,8 \
--out runs/nano2_conv
bash examples/abinit/run_convergence_efg_wsl.sh runs/nano2_conv
PYTHONPATH=src python3 examples/abinit/efg_convergence.py collect \
--workdir runs/nano2_conv --out runs/nano2_conv/convergence.md \
--csv runs/nano2_conv/convergence.csv
```

Each knob's ladder shares one baseline job (only off-baseline values are staged as extra runs), so the baseline is computed once and anchored across all three sweeps. `run_convergence_efg_wsl.sh` runs each job in a WSL-native scratch directory (a `mktemp` dir under `\protect\T1\textdollarTMPDIR`, overridable with `ABINIT_SCRATCH_DIR`) and copies only `<name>.abo` back into the `workdir`. This matters on OneDrive-synced `/mnt/c` checkouts, where ABINIT scratch and `.abo` I/O is slow and OneDrive file locks can stall a run so that no `.abo` ever appears.

NaNO₂ result. For the ¹⁴N site the sweep is decisive: η is flat to $\sim 10^{-4}$ across `pawcutdg` (50 → 100 Ha) and `ngkpt` ($4^3 \rightarrow 8^3$, n_{kpt} up to 260), while `ecut` moves it only slightly and *away* from experiment (knee near 30 Ha). Convergence — and, because the inputs run with `nsym 1`, symmetry pinning — is therefore ruled out as the cause of the low η . The geometry is the dominant lever instead:

geometry	η	C_Q (MHz)
hand-built starter	0.043	−5.17
ICSD 82857 (experimental)	0.112	−5.03
experiment (77–300 K NQR)	~ 0.38	± 5.49

The experimental cell raises η by $\sim 2.6\times$ over the starter geometry but still under-predicts the measured value. The cause of that residual was pinned down with an all-electron cross-check.

All-electron cross-check (Elk): the gap is PAW-sensitive. The transverse EFG components that set η are a near-nucleus quantity, so they are sensitive to how the pseudopotential reconstructs the core region. Repeating the calculation with the *same* PBE functional and the *same* ICSD geometry in the all-electron full-potential LAPW code Elk — the reference method for EFGs — gives a very different η :

method (PBE, ICSD geometry)	η	$ C_Q $ (MHz)
experiment (77–300 K NQR)	~ 0.38	5.49
ABINIT — PAW (Pseudodojo)	0.112	5.03
Elk — all-electron LAPW	≈ 0.333	5.9

All-electron PBE recovers $\eta \approx 0.333$ (converged over `rgkmax` 7 → 8 and the k -mesh, 18 → 48 points), next to the measured 0.38, whereas the PAW value sits $\sim 3\times$ lower. Same functional, same geometry: the

comparison isolates a strong **PAW/pseudopotential sensitivity**. The selected Pseudodojo PBE nitrogen dataset under-represents the near-nucleus transverse EFG in this case; geometry and functional do not explain the ABINIT–Elk difference. (Harmonic vibrational averaging, the finite-temperature workflow above, in fact *lowers* η , so dynamics does not close the gap upward.) The ABINIT-side An ABINIT-side next test is therefore a harder or EFG-tuned nitrogen PAW dataset before changing the functional. The Elk cross-check workflow — input, an EFG.OUT parser, and an MPI-parallel runner with a convergence watchdog — is in `examples/elk/`; its `README.md` records the inputs, convergence results, performance observations, and interpretation.

8 Finite-Strain EFG Drive Couplings

In this chapter. You will generate, collect, and validate the thirteen calculations needed for central finite-strain EFG derivatives and transition-drive couplings.

Piezoelectric NQR drive needs a different DFT derivative from the finite-temperature workflow. Instead of displacing atoms along optical phonon normal modes, the experiment applies a macroscopic strain field through the piezoelectric crystal. To first order,

$$V_{ij}(\epsilon) \simeq V_{ij}(0) + \sum_{\mu} \frac{\partial V_{ij}}{\partial \epsilon_{\mu}} \epsilon_{\mu},$$

where $\mu \in \{xx, yy, zz, yz, xz, xy\}$ is the symmetric strain component in Voigt notation. The transition-drive coefficient is obtained by projecting $\partial V_{ij}/\partial \epsilon_{\mu}$ into the eigenbasis of the equilibrium quadrupolar Hamiltonian. This is the quantity used by the piezoelectric drive/detection model in PythonSpinDynamics.

8.1 Glycine polymorph guard

The bundled glycine structure is CCDC 189379, space group P 21. This non-centrosymmetric polymorph is symmetry-allowed to be piezoelectric, whereas centrosymmetric glycine polymorphs are not useful for an electric-field-driven piezoelectric transducer. The CLI therefore performs a CIF metadata check before writing jobs:

```
PYTHONPATH=src python3 examples/abinit/strain_efg_coupling.py check
```

For CCDC 189379, the expanded ABINIT cell contains two symmetry-related nitrogen sites. The zero-based target indices printed by prepare-base are 1 and 11; the example below uses index 1.

8.2 Static and strained ABINIT jobs

The strain workflow begins by preparing a static EFG input directly from the CIF. On Ubuntu's packaged ABINIT 9.10.4 PAW set, the glycine H/C/N/O XML files are in `Pseudodojo_paw_pw_standard`. Those PAW spheres overlap on short X–H bonds, so the exploratory run used `pawovlp 25`; production calculations should replace this with smaller-radius PAW datasets or a validated pseudopotential choice.

```
PYTHONPATH=src python3 examples/abinit/strain_efg_coupling.py prepare-base \
--out runs/glycine_static/glycine_efg.abi \
--pawovlp 25

bash examples/abinit/run_static_efg_wsl.sh \
runs/glycine_static/glycine_efg.abi
```

After the static input is validated, the generator writes the equilibrium plus central-difference strained structures:

```
PYTHONPATH=src python3 examples/abinit/strain_efg_coupling.py generate \
--base runs/glycine_static/glycine_efg.abi \
--target-atom-index 1 \
--out runs/glycine_strain

bash examples/abinit/run_strain_efg_wsl.sh runs/glycine_strain
```

The collection step parses each .abo, finite-differences the target nucleus EFG tensor, diagonalizes the equilibrium spin-1 Hamiltonian, and writes JSON/CSV coupling tables:

```
PYTHONPATH=src python3 examples/abinit/strain_efg_coupling.py collect \
--workdir runs/glycine_strain \
--quadmom 0.02044 \
--strain-peak 1e-5 \
--json runs/glycine_strain/couplings.json \
--csv runs/glycine_strain/couplings.csv
```

8.3 First-pass glycine result

The exploratory glycine calculation completed all 13 jobs (equilibrium plus $\pm$ strains for six components). The equilibrium EFG at the target nitrogen was

$$(V_{xx}, V_{yy}, V_{zz}) = (-1.9249, -1.2972, 3.2221) \times 10^{21} \text{ V/m}^2, \quad \eta = 0.1948,$$

corresponding to $C_Q = 1.5925 \text{ MHz}$ for ^{14}N . The NQRDatabase glycine amine- ^{14}N entry is $C_Q = 1.193 \text{ MHz}$ and $\eta = 0.528$, with observed lines at 737 and 1052 kHz. Thus the DFT run captures the MHz scale but makes the tensor too axial; absolute frequencies are not yet production quality.

Transition	Best strain	Frequency (kHz)	Coupling (MHz/strain)	Rabi at 10^{-5} strain (Hz)
1 $\rightarrow$ 2	xz	155.124	1.456	7.28
0 $\rightarrow$ 1	yy	1116.801	1.844	9.22
0 $\rightarrow$ 2	xz	1271.924	2.189	10.95

Table 8.1: Strongest projected quadrupolar drive coefficients from the first-pass glycine finite-strain calculation. The Rabi rate assumes a peak acoustic strain of 10^{-5} .

For the experimental two-line glycine model in PythonSpinDynamics, the strongest DFT coefficients are mapped onto the observed lower and upper lines as practical drive estimates: the lower 737 kHz line uses the yy -driven 0 $\rightarrow$ 1 coefficient (1.844 MHz/strain), and the upper 1052 kHz line uses the xz -driven 0 $\rightarrow$ 2 coefficient (2.189 MHz/strain). This keeps the detection model tied to measured frequencies while using DFT-scale electric-field-gradient derivatives.

Reliability checkpoint. These are exploratory couplings, not design numbers. Before using them quantitatively, repeat the workflow with converged `ecut`, `pawecutdg`, and `ngkpt`; relax at least the hydrogen positions or use a neutron-quality structure; verify the symmetry-related nitrogen index 11; test strain-step linearity; and remove or quantify the effect of `pawovlp 25`.

9 Worked Example: NaNO_2 ^{14}N

In this chapter. You will follow the repository's reference system from its source structure through static and finite-temperature results, including the checks that limit what may be concluded.

Sodium nitrite is the reference case for the workspace: its ^{14}N parameters appear in the DFT runs, the spin-dynamics simulator, and the measured NQR database. A six-mode finite-displacement calculation on the starter geometry produces the temperature dependence in Table 9.1.

There are two useful ways to follow this example:

1. use the small starter calculation to learn the mechanics of the staged workflow and recognize its failure checks; then
2. use the ICSD 82857 structure, converged ABINIT calculation, and Elk comparison for the stronger accuracy argument.

Do not combine the first track's convenient teaching numbers with the second track's scientific conclusions.

T (K)	C_Q (MHz)	η	lines (MHz)
static	-5.173	0.043	0.11, 3.82, 3.94
0	-4.963	0.016	0.04, 3.70, 3.74
77	-4.927	0.014	0.03, 3.68, 3.71
150	-4.816	0.002	0.00, 3.61, 3.61
300	-4.534	0.033	0.08, 3.36, 3.44

Table 9.1: Finite-temperature ^{14}N EFG of NaNO_2 from a six-mode harmonic average on the starter geometry. The slope of the two strong lines is $d\nu/dT \approx -1.0$ to -1.1 kHz/K over 0–300 K.

The result reproduces the expected physics with no fitting: $|C_Q|$ and the lines decrease smoothly with temperature (the Bayer sign), the 0 K value already lies below the static one by about 4% (zero-point motion), and η shifts with temperature. The slope, ~ -1.0 kHz/K, has the correct sign and order of magnitude relative to the measured coefficients of -1.3 kHz/K (ν_-) and -2.2 kHz/K (ν_+).

Why this is a teaching result, not a prediction. The numbers above are for the *unrelaxed* starter geometry: its static $\eta = 0.04$ disagrees with the experimental 0.38, the two strong lines come out near-degenerate (~ 3.7 MHz) rather than well split (3.60/4.64 MHz), and the phonon spectrum contains imaginary modes. The temperature trend is physically sensible, but the reference is not mechanically stable. Run Stage 0 (§7.2) and pass `relaxed.abi` as the base input to remove that particular defect. Relaxation alone does not make the absolute EFG accurate; the later structure and Elk comparisons test the remaining sources of error.

Runnable comparison. `examples/abinit/nano2_relaxation_study.sh` automates exactly this check: it runs the full temperature workflow on both the unrelaxed and the relaxed geometry and writes a side-by-side report (static EFG, the sweep, and $d\nu/dT$) scored against the measured NaNO_2 ^{14}N reference. See `examples/abinit/README_relaxation_study.md` for the prerequisites and the one-command invocation.

Relaxation outcome. Running this on the starter geometry confirms the pipeline behaves correctly and is qualitatively improved by relaxation, but also exposes the starter model's limit. Relaxing the internal coordinates removes all imaginary modes (the spectrum becomes dynamically stable, lowest libration $\sim 44\text{ cm}^{-1}$) and fixes the sign of every $d\nu/dT$, and it moves the asymmetry and the line splitting toward experiment — but only about a third of the way ($\eta : 0.04 \rightarrow 0.14$ against the measured 0.38; strong-line splitting $0.11 \rightarrow 0.37\text{ MHz}$ against $\sim 1.04\text{ MHz}$). Because the cell was held fixed at the hand-entered starter (`optcell 0`, unconverged `ecut/pawecutdg/ ngkpt`), this run cannot isolate the residual error. Quantitative work therefore starts with a converged calculation on an experimental structure (e.g. the ICSD 82857 input under `structures/NaNO2/generated/`), followed by the ABINIT–Elk comparison in Chapter 2. For the full slope magnitude, anharmonic (AIMD/PIMD) averaging may also be needed near the ferroelectric transition.

CIF validation outcome. The bundled NaNO_2 CIFs now have a two-stage check. `examples/check_nano2_cifs.py` first ranks the available structures by phase, temperature, occupancy and crystallographic metadata; for the room-temperature ferroelectric NQR reference, ICSD 82857 is the best structural target. `examples/abinit/nano2_cif_efg_validation.sh` then stages the top full-occupancy CIFs, runs static EFG calculations through the MPI-aware ABINIT wrapper, and compares the resulting ^{14}N lines against the measured NaNO_2 reference. In the first validation sweep the best simulated line agreement came from ICSD 15400, but the full-occupancy ferroelectric structures all gave the same qualitative answer: $|C_Q|$ was close, while η remained too small ($\eta \simeq 0.11\text{--}0.12$ versus the measured 0.38).

Interpretation. The NaNO_2 asymmetry error is not fixed by choosing a different bundled CIF. The converged same-geometry, same-functional Elk comparison then isolates the selected nitrogen PAW dataset as the dominant source of the static ABINIT–Elk asymmetry gap. Thermal expansion, anharmonicity, and structural disorder remain separate questions when comparing either static calculation with finite-temperature experiment. High-temperature 1 m m m CIFs should also be treated carefully: the available entries are partial-occupancy/disordered models, and a literal static expansion can place split sites nearly on top of each other, causing ABINIT PAW sphere-overlap aborts. Use those only after constructing an ordered approximant, not as ordinary static EFG inputs.

What the example establishes. The software path preserves tensor conventions and produces the expected thermal sign; relaxation removes the starter structure's imaginary modes; structure screening does not resolve the asymmetry; and the controlled Elk comparison reveals PAW sensitivity. Each statement comes from a different check, and none should be claimed from the temperature table alone.

PART IV

Evidence and Reference

10 Validation Against the Measured Database

In this chapter. You will compare calculated static parameters and temperature coefficients with measured records through the cross-project integration layer.

The cross-project `mr_integration` layer closes the loop against the `NQRDatabase`. `measured_temperature_coefficients` reads the stored $d\nu/dT$ for a compound and isotope; `compare_temperature_coefficients` matches a predicted set of (frequency, $d\nu/dT$) pairs — from either a `efg_temperature_sweep` (via `slopes_from_temperature_points`) or a Bayer fit — to the measured coefficients by frequency, so that a DFT temperature dependence can be checked against experiment. The same layer also validates the static (C_Q, η) against measured line positions and supplies the $C_Q \leftrightarrow \nu_Q$ conversion used by the simulator.

Use that comparison in the following order:

1. select the compound, isotope, phase, and measurement temperature;
2. compare static C_Q , η , and line ordering before comparing slopes;
3. match calculated and measured lines by frequency, then inspect whether that assignment remains stable across temperature; and
4. report residuals together with the DFT convergence and structural provenance, rather than treating the database value as an exact target.

Agreement with experiment tests the complete model, not only the Python conversion. Disagreement can originate in the measured assignment, structure, nuclear data, electronic-structure approximation, or temperature model.

11 Public API Reference

In this chapter. Use this chapter after the narrative workflows when you need to find the Python object or function that implements a particular step.

11.1 `quadrupolar_dft.tensors / quadrupolar / abinit`

- `EFGTensor`, `average_tensors` — tensor conventions and frame-correct averaging.
- `coupling_constant_hz`, `nqr_frequencies_hz` — C_Q and zero-field transition frequencies.
- `AbinitEFGRecord`, `parse_abinit_efg`, `format_abinit_efg_block` — ABINIT EFG input/output.

11.2 `quadrupolar_dft.thermal`

- `thermal_quantum_factor`, `bose_occupation`, `mean_square_normal_coordinate`, `wavenumber_to_angular_frequency`.

11.3 `quadrupolar_dft.vibrational`

- `VibrationalMode`, `efg_curvature_central_difference`, `thermally_averaged_efg`, `efg_temperature_sweep`, `ThermalEFGPoint`.
- `bayer_frequency`, `bayer_slope_hz_per_k`, `fit_bayer_single_mode`, `BayerFit`.

11.4 `quadrupolar_dft.finite_displacement / abinit_phonon`

- `Crystal`, `PhononMode`, `parse_abinit_structure`, `generate_displacement_jobs`, `abinit_input_with_positions`, `write_jobs`.
- `collect_efg_outputs`, `vibrational_modes_from_collected`, `vibrational_modes_from_efg`, `efg_tensor_from_record`.
- `phonon_dfpt_input`, `anadddb_input`, `parse_anadddb_modes`, `modes_from_arrays`.

11.5 `quadrupolar_dft.strain_coupling`

- `generate_strain_jobs`, `write_strain_jobs` — central finite-difference homogeneous strain inputs.
- `collect_strain_derivatives` — parse strained EFG outputs and compute $\partial V_{ij} / \partial \epsilon_\mu$.
- `strain_transition_couplings` — project strain derivatives onto spin-1 quadrupolar transition matrix elements.

- `glycine_static_efg_input_from_cif`, `cif_structure_metadata`, `space_group_is_likely_centrosymmetric` — glycine setup and polymorph guards for piezoelectric work.

11.6 quadrupolar__dft.relax

- `relax_input` (ionic-relaxation input from a static EFG input), `parse_relaxed_structure` (relaxed geometry from the ABINIT output footer), `relaxed_input` (relaxed static EFG input for Stages 1–2).

12 Limits of the Current Workflow

In this chapter. You will distinguish implemented, validated capabilities from known model limitations and choose an appropriate next validation step.

The repository automates a substantial calculation chain, but it does not automate scientific judgment. The following boundaries should appear in any report that relies on the workflow:

- The $C_Q \leftrightarrow \nu_Q$ scale and several helpers are calibrated for spin 1 and spin 3/2; higher half-integer spins are not yet handled by the simulator-facing conversion.
- The generated DFPT input is a template; PAW DFPT settings, tolerances and k -mesh must be checked for production accuracy.
- The anaddb eigenvector parser targets ABINIT 9.x text output; a `--modes` JSON escape hatch covers other layouts.
- Finite-difference curvatures use the printed EFG precision; large displacement amplitudes or extra output digits may improve the second derivative, but step-size stability must still be demonstrated.
- The harmonic finite-temperature calculation does not include quasi-harmonic cell expansion or anharmonic sampling.
- Elk support is a static reference workflow, not a replacement backend for the ABINIT DFPT, displacement, and strain automation.

Choosing a method beyond these limits

Use temperature-dependent experimental structures or a quasi-harmonic treatment when cell expansion is important. Use AIMD/PIMD snapshot averaging when anharmonicity or disorder invalidates the normal-mode picture. Convert other phonon formats explicitly when the ANADDB text parser is unsuitable. In each case, keep the downstream tensor conventions unchanged: average full tensors in a common frame, then diagonalize and calculate the NQR observables.

A good stopping point. A calculation is complete when it answers the stated scientific question with documented numerical, structural, and model uncertainty. Adding a more expensive method is useful only when it tests an uncertainty that matters to that answer.